

WINE QUALITY CLASSIFICATION

DATASET TAKEN FROM: Kaggle

CODE:

Import libraries and get basic information – head, info:

```
[44]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

[45]: data = pd.read_csv("winequalityN.csv")
data.head()
```

	type	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
0	white	7.0	0.27	0.36	20.7	0.045	45.0	170.0	1.0010	3.00	0.45	8.8	6
1	white	6.3	0.30	0.34	1.6	0.049	14.0	132.0	0.9940	3.30	0.49	9.5	6
2	white	8.1	0.28	0.40	6.9	0.050	30.0	97.0	0.9951	3.26	0.44	10.1	6
3	white	7.2	0.23	0.32	8.5	0.058	47.0	186.0	0.9956	3.19	0.40	9.9	6
4	white	7.2	0.23	0.32	8.5	0.058	47.0	186.0	0.9956	3.19	0.40	9.9	6

Describe():

```
data.info()
data.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6497 entries, 0 to 6496
Data columns (total 13 columns):
#   Column              Non-Null Count  Dtype
---  -
0    type                 6497 non-null   object
1    fixed acidity         6487 non-null   float64
2    volatile acidity      6489 non-null   float64
3    citric acid           6494 non-null   float64
4    residual sugar        6495 non-null   float64
5    chlorides             6495 non-null   float64
6    free sulfur dioxide   6497 non-null   float64
7    total sulfur dioxide  6497 non-null   float64
8    density              6497 non-null   float64
9    pH                   6488 non-null   float64
10   sulphates            6493 non-null   float64
11   alcohol              6497 non-null   float64
12   quality              6497 non-null   int64
dtypes: float64(11), int64(1), object(1)
memory usage: 660.0+ KB
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
count	6487.000000	6489.000000	6494.000000	6495.000000	6495.000000	6497.000000	6497.000000	6497.000000	6488.000000	6493.000000	6497.000000	6497.000000
mean	7.216579	0.339691	0.318722	5.444326	0.056042	30.525319	115.744574	0.994697	3.218395	0.531215	10.491801	5.818378
std	1.296750	0.164649	0.145265	4.758125	0.035036	17.749400	56.521855	0.002999	0.160748	0.148814	1.192712	0.873255
min	3.800000	0.080000	0.000000	0.600000	0.009000	1.000000	6.000000	0.987110	2.720000	0.220000	8.000000	3.000000
25%	6.400000	0.230000	0.250000	1.800000	0.038000	17.000000	77.000000	0.992340	3.110000	0.430000	9.500000	5.000000
50%	7.000000	0.290000	0.310000	3.000000	0.047000	29.000000	118.000000	0.994890	3.210000	0.510000	10.300000	6.000000
75%	7.700000	0.400000	0.390000	8.100000	0.065000	41.000000	156.000000	0.996990	3.320000	0.600000	11.300000	6.000000
max	15.900000	1.580000	1.660000	65.800000	0.611000	289.000000	440.000000	1.038980	4.010000	2.000000	14.900000	9.000000

Handling missing values:

```
data.isnull().sum()
```

```
type                0
fixed acidity       10
volatile acidity     8
citric acid         3
residual sugar      2
chlorides           2
free sulfur dioxide  0
total sulfur dioxide 0
density            0
pH                 9
sulphates           4
alcohol             0
quality            0
dtype: int64
```

Removing unnecessary columns:

```
data = data.dropna()
```

```
if 'type' in data.columns:
    data = data.drop('type', axis=1)
data.head()
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
0	7.0	0.27	0.36	20.7	0.045	45.0	170.0	1.0010	3.00	0.45	8.8	6
1	6.3	0.30	0.34	1.6	0.049	14.0	132.0	0.9940	3.30	0.49	9.5	6
2	8.1	0.28	0.40	6.9	0.050	30.0	97.0	0.9951	3.26	0.44	10.1	6
3	7.2	0.23	0.32	8.5	0.058	47.0	186.0	0.9956	3.19	0.40	9.9	6
4	7.2	0.23	0.32	8.5	0.058	47.0	186.0	0.9956	3.19	0.40	9.9	6

Convert numbers to Categories

```
def convert(q):
    if q <= 4:
        return 0
    elif q <= 6:
        return 1
    else:
        return 2
data['quality_class'] = data['quality'].apply(convert)
```

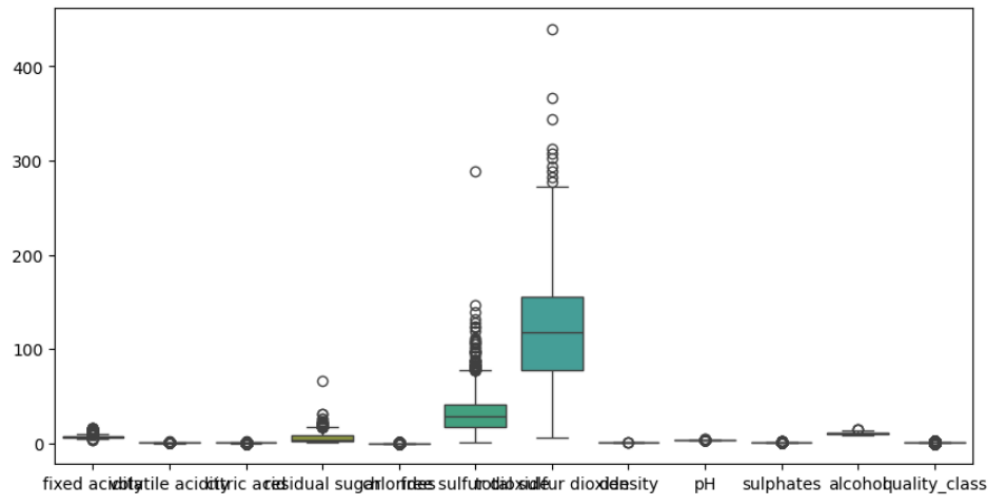
```
data = data.drop('quality', axis=1)
```

```
data['quality_class'].value_counts()
```

```
quality_class
1    4948
2    1271
0     244
Name: count, dtype: int64
```

Visualize outliers:

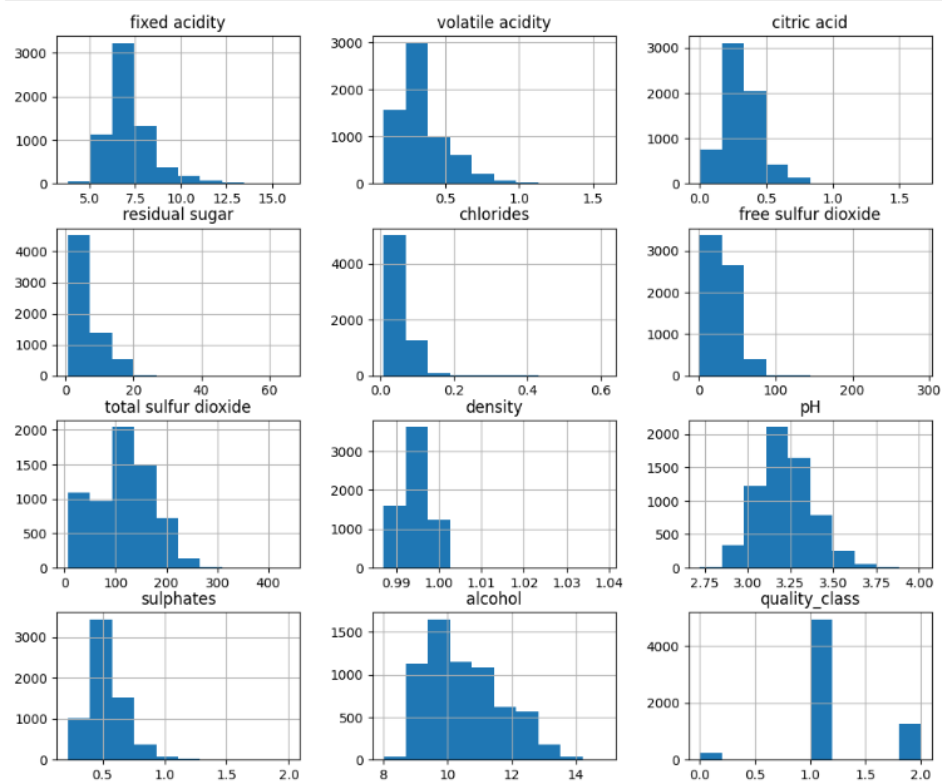
```
plt.figure(figsize=(10,5))
sns.boxplot(data=data)
plt.show()
```



Plots:

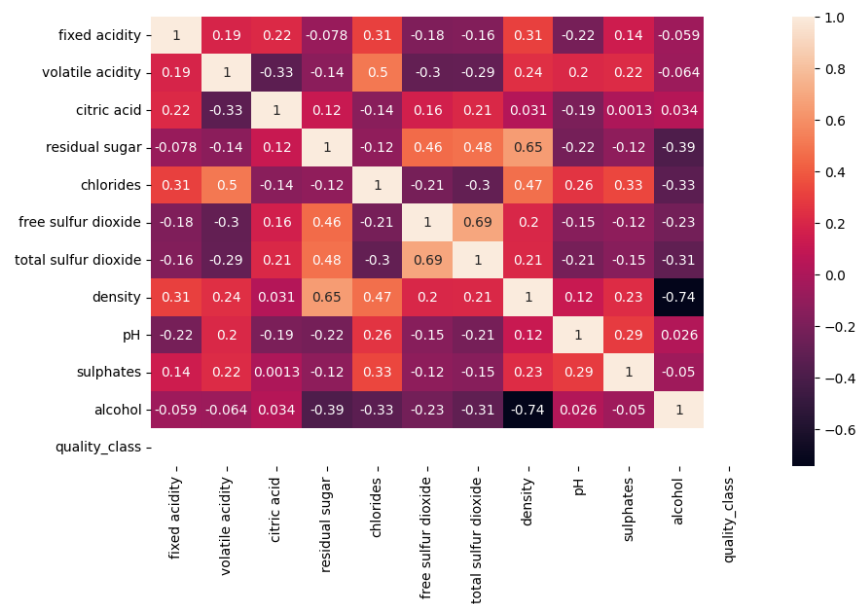
Histogram:

```
data.hist(figsize=(12,10))
plt.show()
```



Correlation Heatmap:

```
plt.figure(figsize=(10,6))
sns.heatmap(data.corr(), annot=True)
plt.show()
```



Model Building:

```
X = data.drop('quality_class', axis=1)
```

```
y = data['quality_class']
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, stratify=y
```

```
)
```

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)
```

Evaluation:

Classification

Logistic Regression:

```
lr = LogisticRegression(max_iter=1000)
lr.fit(X_train, y_train)
pred_lr = lr.predict(X_test)
acc_lr = accuracy_score(y_test, pred_lr)
print("Logistic Regression Accuracy:", acc_lr)
```

```
Logistic Regression Accuracy: 0.794276875483372
```

KNN:

```
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
pred_knn = knn.predict(X_test)
acc_knn = accuracy_score(y_test, pred_knn)
print("KNN Accuracy:", acc_knn)
```

KNN Accuracy: 0.814385150812065

Comparing Logistic Regression and KNN:

```
if acc_lr > acc_knn:
    print("Logistic Regression is better")
    final_pred = pred_lr
else:
    print("KNN is better")
    final_pred = pred_knn
```

KNN is better

Prediction:

```
sample = [X_test[0]]
print("Actual:", y_test.iloc[0])
print("LR Prediction:", lr.predict(sample))
print("KNN Prediction:", knn.predict(sample))
```

Actual: 1

LR Prediction: [1]

KNN Prediction: [1]

Final Accuracy Comparison:

```
print("\nFinal Accuracy Comparison")
print("Logistic Regression:", acc_lr)
print("KNN:", acc_knn)
```

Final Accuracy Comparison

Logistic Regression: 0.794276875483372

KNN: 0.814385150812065

Algorithm Used in this Wine Quality Classification:

Two machine learning classification algorithms are used:

- Logistic Regression
- K-Nearest Neighbors (KNN)

These algorithms are applied to classify wine quality into three categories:

- Low
- Medium
- High

Why you chose it:

Why I'm choose the LR and KNN means,

Logistic Regression is chosen because it is a simple and efficient algorithm for classification problems. It works well when the relationship between features and target is linear.

K-Nearest Neighbors (KNN) is chosen because it classifies data based on similarity between samples. It performs well when similar data points belong to the same class.

Using both algorithms helps in comparing performance and selecting the best model

Result obtained:

Both models were trained using the training dataset and evaluated using the test dataset.

The performance of each model was measured using accuracy.

Among the two, the model with higher accuracy was selected as the best model for predicting wine quality.

The final model was able to successfully classify wine samples into Low, Medium, and High categories.